Ex. No.: 4	Single Layer Perceptron
Date: 29.01.2026	

Aim

To implement a Single Layer Perceptron model for binary classification using Python, perform preprocessing on the dataset, train the perceptron using the perceptron learning rule, and evaluate the model performance using appropriate metrics.

Youtube video link: https://youtu.be/2Nch9jGmJdw

Description

A Perceptron is one of the simplest types of Artificial Neural Networks used for solving binary classification problems. It was introduced by Frank Rosenblatt in 1957 and forms the basic building block of modern neural networks. A Single Layer Perceptron (SLP) consists of:

- Input layer
- Weights
- Bias
- Activation function
- Output layer

Each input feature is multiplied by a weight, and the weighted inputs are summed together along with a bias term. The perceptron computes the output using the following equation:

$$Z = w1x1 + w2x2 + \ldots + wnxn + b$$

Where

- x1,x2,...,xn are input features
- w1,w2,...,wn are weights
- bbb is the bias

The result is passed through a step activation function to produce the final output:

$$f(z) = \begin{cases} 1, & z \geq 0 z < 0 \\ 0, & z \geq 0 z < 0 \end{cases}$$

The perceptron learns by adjusting its weights using the Perceptron Learning Rule**:**

$$\mathbf{w} = \mathbf{w} + \boldsymbol{\eta} \times (\mathbf{y} - \overline{\boldsymbol{y}}) \times \mathbf{x}$$

Where

- η\etaη is the learning rate
- yyy is the actual output
- y^\hat{y}y^ is the predicted output

The model iteratively updates the weights until the classification error is minimized.

Program

Question 1
Read CSV data into pandas dataframe object

Code
```
print("Regno: URK23CS1263\n")
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import accuracy_score, f1_score, precision_score, recall_score,
classification_report, confusion_matrix
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
df = pd.read_csv("penguins_SLP.csv")
df.head()
```
Output

Regno: URK23CS1263

	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	species
0	39.1	18.7	181.0	3750.0	Adelie
1	39.5	17.4	186.0	3800.0	Adelie
2	40.3	18.0	195.0	3250.0	Adelie
3	NaN	NaN	NaN	NaN	Adelie
4	36.7	19.3	193.0	3450.0	Adelie

Question 2
Do necessary preprocessing [data imputation with mean for null values, label encoding to convert categorical to numerical]
Code
```
print("Regno: URK23CS1263\n")
df.fillna(df.mean(numeric_only=True), inplace=True)
le = LabelEncoder()
df[target] = le.fit_transform(df[target])
print(df.isnull().sum())
df[target]
```

Output

```
Regno: URK23CS1263

bill_length_mm       0
bill_depth_mm        0
flipper_length_mm    0
body_mass_g          0
species              0
dtype: int64

0     0
1     0
2     0
3     0
4     0
     ..
95    1
96    1
97    1
98    1
99    1
Name: species, Length: 100, dtype: int64
```

Question 3

Choose independent variables (X1, X2 ....) and dependent or target variable (Y) from given Dataset

Code

```
print("Regno: URK23CS1263\n")
X = df[features].values
y = df[target].values
X.shape, y.shape
```

Output

```
Regno: URK23CS1263


((100, 4), (100,))
```

Question 4

Standardize the input features

Code

```
print("Regno: URK23CS1263\n")
X = (X - X.mean(axis=0)) / X.std(axis=0)
print(X[0:5])
```

Output

```
Regno: URK23CS1263

[[-7.91353601e-01  8.57644476e-01 -1.31900139e+00 -7.47886468e-01]
 [-7.08903226e-01  2.88094197e-01 -9.93239961e-01 -6.88055550e-01]
 [-5.44002476e-01  5.50963556e-01 -4.06869382e-01 -1.34619564e+00]
 [-1.46461288e-15  0.00000000e+00  1.85173936e-15  0.00000000e+00]
 [-1.28605585e+00  1.12051384e+00 -5.37173955e-01 -1.10687197e+00]]
```

Question 5

Split the data into training and testing

Code

```
print("Regno: URK23CS1263\n")
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
print("Training and Testing data is splited.")
```

Output

```
Regno: URK23CS1263

Training and Testing data is splited.
```

Question 6

Assign the learning rate, number of iterations, initial weights, and bias

Code

```
print("Regno: URK23CS1263\n")
earning_rate = 0.01
iterations = 100
n_samples, n_features = X_train.shape
n_classes = len(np.unique(y))
weights = np.zeros((n_classes, n_features))
bias = np.zeros(n_classes)
errors = []
print("learning rate, number of iterations, initial weights, and bias are assigned.")
```

Output

```
Regno: URK23CS1263

learning rate, number of iterations, initial weights, and bias are assigned.
```

Question 7

Calculate the updated weight in every iteration during training

Code

```
print("Regno: URK23CS1263\n")
for it in range(iterations):
    total_error = 0
    for idx, x_i in enumerate(X_train):
        linear_output = np.dot(weights, x_i) + bias
        y_pred = np.argmax(linear_output)
        if y_pred != y_train[idx]:
            total_error += 1
            weights[y_train[idx]] += learning_rate * x_i
            bias[y_train[idx]] += learning_rate
            weights[y_pred] -= learning_rate * x_i
            bias[y_pred] -= learning_rate
    errors.append(total_error)
print("updated weights: ")
weights
```

Output

```
Regno: URK23CS1263

updated weights:
array([[-0.00494702,  0.02278201, -0.01693959, -0.01435942],
       [ 0.00494702, -0.02278201,  0.01693959,  0.01435942]])
```

Question 8

Display the final weight of each input attribute after the training

Code

```
print("NIVESH R   URK23CS1262\n")
print("--- Final Model Weights ---")
features = ['bill_length', 'bill_depth', 'flipper_length', 'body_mass']
for i, w in enumerate(weights):
    print(f"{features[i]}: {w:.4f}")
print(f"Final Bias: {bias:.4f}\n")
```

Output

```
Regno: URK23CS1263

Final Weights After Training:

Class: 0
  bill_length_mm: -0.004947022513162265
  bill_depth_mm: 0.022782011151503065
  flipper_length_mm: -0.016939594523081806
  body_mass_g: -0.014359420180313767
  Bias: 0.01

Class: 1
  bill_length_mm: 0.004947022513162265
  bill_depth_mm: -0.022782011151503065
  flipper_length_mm: 0.016939594523081806
  body_mass_g: 0.014359420180313767
  Bias: -0.01
```

Question 9

Plot the graph for iteration against error during the training

Code

```
print("Regno: URK23CS1263\n")
plt.figure()
plt.plot(range(iterations), errors)
plt.xlabel("Iterations")
plt.ylabel("Number of Misclassifications")
plt.title("Iteration vs Training Error")
plt.show()
```

Output

```
Regno: URK23CS1263
```

Question 10

Predict the class labels for the test set

Code

```
print("Regno: URK23CS1263\n")
def predict(X):
    linear_output = np.dot(X, weights.T) + bias
    return np.argmax(linear_output, axis=1)
y_pred_test = predict(X_test)
y_pred_test
```

Output

```
Regno: URK23CS1263


array([1, 1, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 1, 0, 1, 1, 0, 0, 1, 1,
       0, 0, 1, 0, 0, 1, 0, 1])
```

Question 11

Display the confusion matrix and calculate the performance metrics such as accuracy, precision, recall, and f1_score of the model

Code

```
print("Regno: URK23CS1263\n")
print("\nConfusion Matrix:")
cm = confusion_matrix(y_test, y_pred_test)
print(cm)
accuracy = accuracy_score(y_test, y_pred_test)
precision = precision_score(y_test, y_pred_test, average='weighted')
recall = recall_score(y_test, y_pred_test, average='weighted')
f1 = f1_score(y_test, y_pred_test, average='weighted')
print("\nPerformance Metrics:")
print("Accuracy :", accuracy)
print("Precision:", precision)
print("Recall   :", recall)
```

```
print("F1 Score :", f1)
```

Output

```
Regno: URK23CS1263

Confusion Matrix:
[[17  0]
 [ 0 13]]

Performance Metrics:
Accuracy : 1.0
Precision: 1.0
Recall   : 1.0
F1 Score : 1.0
```

Result

The Single Layer Perceptron model was successfully implemented and trained on the Iris dataset.